

Практическая работа

Двусвязные списки. Деки

В двусвязном списке на каждый элемент списка указывает два указателя (из предыдущего элемента и из следующего), каждый элемент имеет два указателя — на предыдущий элемент и на следующий, и в списке присутствуют две цепочки связности — прямая и обратная. Обычно с таким списком работают с помощью двух указателей — на первый его элемент и на последний.

Построить двусвязный список можно из звеньев, описанных, например, так:

```
type
  item2ptr = ^item2;
  item2 = record
    data: integer;
    prev, next: item2ptr;
  end;
```

Как и в случае со словом `next` для указателя на следующий элемент, слово `prev` (от англ. *previous*) традиционно обозначает указатель на предыдущий; конечно, вам никто не мешает использовать другое название, но если вашу программу будет читать кто-то ещё, то понять её окажется сложнее.

Двусвязные списки применяются несколько реже, чем односвязные, поскольку требуют вдвое больше действий с указателями при любых операциях вставки и удаления звеньев, да и сами звенья за счёт наличия двух указателей вместо одного становятся больше, т. е. занимают больше памяти. С другой стороны, у двусвязных списков имеется целый ряд несомненных достоинств в сравнении с односвязными. Прежде всего, это очевидная симметрия, которая позволяет просматривать список как в прямом направлении, так и в обратном; в некоторых задачах это оказывается важно.

Второе несомненное достоинство двусвязного списка не столь очевидно, но иногда оказывается даже важнее: зная адрес любого из звеньев двусвязного списка, мы можем найти в памяти все его звенья.

Если на первое, последнее и текущее звенья двусвязного списка указывают соответственно указатели `first`, `last` и `current`, новое число находится в переменной `n`, а временный указатель называется `tmp`, вставка нового звена слева от текущего выглядит так:

```
new(tmp);
tmp^.prev := current^.prev;
tmp^.next := current;
tmp^.data := n;
if current^.prev = nil then
  first := tmp
else
  current^.prev^.next := tmp;
current^.prev := tmp;
```

Вставка справа выглядит аналогично:

```

new(tmp);
tmp^.prev := current;
tmp^.next := current^.next;
tmp^.data := n;
if current^.next = nil then
    last := tmp
else
    current^.next^.prev := tmp;
current^.next := tmp;

```

Удаление текущего элемента делается и того проще:

```

if current^.prev = nil then
    first := current^.next
else
    current^.prev^.next := current^.next;
if current^.next = nil then
    last := current^.prev
else
    current^.next^.prev := current^.prev;
dispose(current);

```

Этот фрагмент обладает некоторым недостатком: указатель `current` после его выполнения указывает на несуществующий (уничтоженный) элемент. Этого можно избежать, если воспользоваться временным указателем для удаления элемента, а в `current` предварительно (до уничтожения текущего элемента) занести адрес предыдущего или следующего, в зависимости от того, проходим мы список в прямом направлении или в обратном.

Добавление элемента в начало двусвязного списка с учётом возможного особого случая может выглядеть, например, так:

```

new(tmp);
tmp^.data := n;
tmp^.prev := nil;
tmp^.next := first;
if first = nil then
    last := tmp
else
    first^.prev := tmp;
first := tmp;

```

Добавление в конец производится аналогично (с точностью до замены направления):

```

new(tmp);
tmp^.data := n;
tmp^.prev := last;

```

```

tmp^.next := nil;
if last = nil then
    first := tmp
else
    last^.next := tmp;
last := tmp;

```

Можно обобщить вышеприведённую процедуру вставки слева от текущего звена, предположив, что если текущего нет (то есть вставка происходит «слева от несуществующего звена»), то это вставка в конец списка. Выглядеть это будет так:

```

new(tmp);
if current = nil then
    tmp^.prev := last
else
    tmp^.prev := current^.prev;
tmp^.next := current;
tmp^.data := n;
if tmp^.prev = nil then
    first := tmp
else
    tmp^.prev^.next := tmp;
if tmp^.next = nil then
    last := tmp
else
    tmp^.next^.prev := tmp;

```

Аналогичное обобщение вставки справа от текущего звена на ситуацию «вставка справа от несуществующего есть вставка в начало» происходит практически так же, меняются только строки, отвечающие за заполнение полей tmp^.prev и tmp^.next (в нашем фрагменте это строки со второй по шестую):

```

tmp^.prev := current;
if current = nil then
    tmp^.next := first
else
    tmp^.next := current^.next;

```

Задание 1. Самостоятельно нарисовать на бумаге схематические диаграммы изменения указателей для всех рассмотренных выше случаев.

Двусвязные списки позволяют создать объект, обычно называемый **декой** (англ. Deque, сокр. от **Double-Ended QUEUE**). Дек — это абстрактный объект, поддерживающий четыре операции: добавление в начало, добавление в конец, извлечение из начала и извлечение из конца. Значение, добавленное в начало дека, можно сразу же извлечь обратно (применив извлечение из начала), но если извлекать значения из конца, то значение, только что добавленное в начало, будет извлечено после всех остальных зна-

чений, хранившихся в деке; с добавлением в конец и извлечением из начала ситуация симметричная.

При использовании только операций «добавление в начало» и «извлечение из начала» дек превращается в стек (как, впрочем, и при использовании добавления и извлечения из конца) или LIFO, а при использовании «добавления в начало» и «извлечения из конца» — в очередь или FIFO; но использовать дек в качестве стека или обычной очереди не стоит, ведь для стека и очереди возможны гораздо более простые реализации: их можно реализовать через односвязный список, тогда как дек односвязным списком реализовать нельзя (точнее, можно использовать два таких списка, но это очень неудобно). Отметим, что по-английски соответствующие операции обычно называются pushfront, pushback, popfront и popback.

Начать реализацию дека можно, как обычно, с заглушки. К примеру, для дека, хранящего числа типа longint, заглушка будет такая (условимся, как и раньше, считать, что перед обращением к процедурам извлечения элемента всегда в обязательном порядке производится проверка, не пуст ли наш объект):

```
type
  LongItem2Ptr = ^LongItem2;
  LongItem2 = record
    data: longint;
    prev, next: LongItem2Ptr;
  end;
  LongDeque = record
    first, last: LongItem2Ptr;
  end;

procedure DequeInit(var deque: LongDeque);
begin
end;

procedure DequePushFront(var deque: LongDeque; n: longint);
begin
end;

procedure DequePushBack(var deque: LongDeque; n: longint);
begin
end;

procedure DequePopFront(var deque: LongDeque; var n: longint);
begin
end;

procedure DequePopBack(var deque: LongDeque; var n: longint);
begin
end;
```

```
function DequeIsEmpty(var deque: LongDeque) : boolean;  
begin  
end;
```

Задание 2. Создайте программу Deque, работающую с динамической структурой дек неограниченного объёма. Программа должна считывать команды работы с деком с параметрами и без, выполнять команды, подтверждая их выводом слова Ok и результатом в отдельных строках. Список команд:

push_front `целое_значение`

Добавить в начало дека новый элемент с содержимым «целое_значение». Вывести Ok в отдельной строке.

push_back `целое_значение`

Добавить в конец дека новый элемент с содержимым «целое_значение». Вывести Ok в отдельной строке.

pop_front

Извлечь из дека первый элемент. Вывести его значение в отдельной строке.

pop_back

Извлечь из дека последний элемент. Вывести его значение в отдельной строке.

front

Узнать значение первого элемента (не удаляя его). Программа должна вывести его значение.

back

Узнать значение последнего элемента (не удаляя его). Программа должна вывести его значение.

size

Вывести количество элементов в деке.

clear

Очистить дек (удалить из него все элементы) и вывести ok.

exit

Программа должна вывести bye и завершить работу.

Размер дека должен быть ограничен только размером доступной оперативной памяти. Перед исполнением операций `pop_front`, `pop_back`, `front`, `back` программа должна проверять, содержится ли в деке хотя бы один элемент. Если во входных данных встречается операция `pop_front`, `pop_back`, `front`, `back`, и при этом дек пуст, то программа должна вместо числового значения вывести строку `error`.

Входные данные

Вводятся команды управления деком, по одной на строке.

Выходные данные

Требуется вывести протокол работы дека, по одному сообщению на строке.

Примеры

Входные данные

push_back 1

back

exit

Выходные данные

ok

1

bye

Входные данные

size

push_back 1

size

push_back 2

size

push_front 3

size

exit

Выходные данные

0

ok

1

ok

2

ok

3

bye